

PRD — Plataforma No-Code de Design WebGL Baseada em Camadas

1. Resumo Executivo

1.1 Nome Provisório do Produto

ShaderCanvas Studio

Nome temporário para fins de documentação. O produto final deverá receber naming próprio antes da fase comercial.

1.2 Visão do Produto

Criar uma plataforma de design visual baseada em navegador que permita a designers, motion designers, artistas digitais e desenvolvedores criarem experiências WebGL interativas, animadas e performáticas sem escrever código GLSL.

O produto combina uma experiência familiar de ferramentas como Figma, Photoshop e After Effects com um motor gráfico em tempo real baseado em camadas, shaders, efeitos generativos, máscaras, animações e exportação para web.

A proposta é transformar a criação de cenas WebGL, normalmente restrita a desenvolvedores técnicos, em um fluxo visual acessível, criativo e produtivo.

1.3 Problema

Criar experiências WebGL avançadas exige, hoje, conhecimento técnico em:

- GLSL;
- pipelines gráficos;
- render targets;
- otimização de draw calls;
- textura, UV, máscaras e blending;
- integração com sites e frameworks front-end.

Isso cria uma barreira alta para designers e equipes criativas que querem produzir experiências visuais modernas para landing pages, portfólios, sites institucionais, instalações digitais, marcas e produtos interativos.

1.4 Solução

Uma ferramenta no-code de design WebGL com:

- canvas visual em tempo real;

- sistema de camadas;
- biblioteca de efeitos shader configuráveis;
- animações e interatividade nativas;
- exportação para embed web;
- SDK leve para produção;
- sistema avançado de otimização via flattening;
- estimador de performance por cena e por camada.

1.5 Posicionamento

O produto deve ser entendido como uma ferramenta independente inspirada na categoria de design WebGL no-code. O desenvolvimento não deve copiar código, marca, identidade visual, assets proprietários ou fluxos protegidos de terceiros. O objetivo é construir uma solução própria com princípios semelhantes: criação visual, camadas, shaders e publicação web performática.

2. Objetivos do Produto

2.1 Objetivos Principais

1. Permitir que usuários não técnicos criem cenas WebGL complexas visualmente.
2. Reduzir drasticamente a dependência de código GLSL para criar efeitos interativos.
3. Criar um fluxo de trabalho visual baseado em camadas, máscaras, efeitos e parâmetros.
4. Publicar cenas otimizadas em sites externos com SDK leve.
5. Garantir boa performance em desktop e mobile por meio de flattening, downsampling e estimador de custo.
6. Criar uma ferramenta suficientemente flexível para uso em landing pages, hero sections, backgrounds animados, arte generativa, portfolios e experiências interativas.

2.2 Objetivos de Negócio

1. Lançar um MVP funcional para designers e desenvolvedores front-end.
2. Criar uma base inicial de usuários criativos interessados em WebGL no-code.
3. Validar monetização via planos SaaS.
4. Diferenciar o produto pela combinação de criatividade visual e performance de produção.
5. Possibilitar integração simples com Framer, Webflow, React, Next.js e sites estáticos.

2.3 Objetivos Técnicos

1. Criar um motor WebGL2 modular.
 2. Implementar um sistema de camadas renderizadas sequencialmente.
 3. Criar um runtime SDK leve para embed.
 4. Implementar exportação de cenas como JSON serializado.
 5. Implementar flattening parcial no MVP e flattening avançado em versões futuras.
 6. Criar infraestrutura para presets, templates e biblioteca de efeitos.
-

3. Não Objetivos

O produto não terá, na primeira fase:

- editor GLSL completo para usuários finais;
- substituição completa de ferramentas como Blender, After Effects ou Figma;
- renderização 3D avançada com física, rigging ou animação esquelética;
- colaboração multiplayer em tempo real;
- marketplace público de templates;
- timeline avançada estilo After Effects;
- suporte completo a WebGPU;
- sistema de plugins de terceiros;
- edição vetorial complexa;
- edição de vídeo profissional.

Esses itens podem ser considerados em fases futuras, mas não fazem parte do escopo inicial.

4. Público-Alvo

4.1 Personas

Persona 1 — Designer Visual / Motion Designer

Perfil:

Designer que cria sites, landing pages, campanhas digitais, identidades visuais e peças animadas.

Necessidades:

- criar visuais modernos sem depender de dev;
- experimentar efeitos rapidamente;
- exportar animações para web;
- trabalhar com camadas, máscaras e presets;
- gerar resultados visualmente impactantes.

Dores:

- não sabe GLSL;
 - ferramentas atuais são técnicas demais;
 - exportar vídeo limita a interatividade;
 - experiências WebGL personalizadas custam caro.
-

Persona 2 — Desenvolvedor Front-end Criativo

Perfil:

Dev que trabalha com React, Next.js, Framer, Webflow ou sites institucionais.

Necessidades:

- inserir cenas interativas em sites;
- controlar variáveis via código;
- manter performance em produção;
- ter embed limpo e SDK leve;
- customizar animações e eventos.

Dores:

- criar shaders do zero consome tempo;
 - otimizar WebGL manualmente é complexo;
 - assets criativos precisam de muitas iterações;
 - designers nem sempre conseguem entregar materiais prontos para web.
-

Persona 3 — Estúdio Criativo / Agência**Perfil:**

Equipe que entrega sites, campanhas e ativações digitais para marcas.

Necessidades:

- criar experiências premium;
- reutilizar templates;
- publicar cenas rapidamente;
- colaborar entre design e desenvolvimento;
- entregar assets otimizados para clientes.

Dores:

- dependência de especialistas;
 - prazos curtos;
 - alto custo de prototipação;
 - dificuldade de reaproveitar efeitos entre projetos.
-

5. Casos de Uso

5.1 Hero Section Interativa

Usuário cria uma cena WebGL para o topo de uma landing page com ruído generativo, partículas, distorção ao mouse e bloom. Exporta o embed para Framer ou Next.js.

5.2 Background Animado para Marca

Designer cria uma composição abstrata com gradientes, wisps, noise fill, máscaras e movimentos suaves. Exporta como embed ou vídeo.

5.3 Experiência Scroll-Based

Dev cria uma cena em que propriedades de camadas mudam conforme o usuário rola a página. A cena é publicada via SDK com variáveis controladas pelo scroll.

5.4 Efeito de Imagem Distorcida

Usuário importa uma imagem, aplica displacement, swirl, grain, chromatic aberration e máscara de texto. Exporta como imagem estática, vídeo ou cena interativa.

5.5 Template Reutilizável

Estúdio cria presets internos para marcas, com parâmetros editáveis de cor, velocidade, textura e intensidade.

6. Escopo do MVP

6.1 MVP — Objetivo

Entregar uma versão funcional que permita criar, editar, visualizar, salvar e publicar cenas WebGL simples a intermediárias, com camada visual, efeitos configuráveis, animação básica e embed otimizado.

6.2 Funcionalidades Incluídas no MVP

Editor

- Canvas WebGL central.
- Painel de camadas.
- Painel de propriedades.
- Adição, remoção, reordenação e duplicação de camadas.
- Visibilidade e bloqueio de camadas.
- Seleção visual de camada no canvas.
- Transformações 2D básicas: posição, escala, rotação e opacidade.
- Sistema de blend modes inicial.

Camadas

- Shape layer.
- Image layer.
- Video layer.
- Text layer básica.

- Solid color layer.
- Generative shader layer.

Efeitos

MVP deve incluir de 12 a 20 efeitos iniciais, entre eles:

- Noise Fill;
- FBM Distortion;
- Swirl;
- Polar Distortion;
- Wave;
- Gradient;
- Vignette;
- Bloom;
- Blur;
- Dither;
- CRT Scanlines;
- Chromatic Aberration;
- Grain;
- Displacement Map;
- Mask Reveal;
- Color Shift.

Interatividade

- Mouse position.
- Mouse velocity.
- Scroll progress.
- Time.
- Viewport size.
- Runtime variables via SDK.

Animação

- Aparecer/desaparecer;
- delay;
- duração;
- easing;
- loop;
- animação por propriedade;
- keyframes simples para propriedades numéricas.

Exportação

- Exportar cena como JSON.
- Embed code com `<script>`.
- SDK JavaScript.

- Exportar imagem estática PNG/JPEG.
- Exportar vídeo curto em WebM ou MP4, se tecnicamente viável no navegador.

Performance

- Downsampling por camada.
 - Estimador inicial de custo.
 - Flattening básico para grupos compatíveis.
 - Indicador de FPS.
 - Alerta de cena pesada.
-

7. Funcionalidades Pós-MVP

7.1 V1

- Flattening avançado.
- Máscaras compostas.
- Biblioteca expandida com 75+ efeitos.
- Templates.
- Presets por efeito.
- Timeline melhorada.
- Exportação otimizada para Framer/Webflow.
- Controles responsivos por breakpoint.
- Histórico de versões.
- Autenticação e dashboard de projetos.

7.2 V2

- Colaboração em tempo real.
 - Marketplace de templates e efeitos.
 - Editor visual de nós para criação de shaders.
 - Suporte parcial a WebGPU.
 - Importação de modelos 3D mais avançada.
 - Publicação via CDN própria.
 - Analytics de performance das cenas publicadas.
 - API para automação de render/export.
-

8. Requisitos Funcionais

8.1 Gerenciamento de Projetos

ID	Requisito	Prioridade
RF-001	Usuário deve poder criar um novo projeto.	P0

ID	Requisito	Prioridade
RF-002	Usuário deve poder salvar projeto na nuvem.	P0
RF-003	Usuário deve poder renomear projeto.	P0
RF-004	Usuário deve poder duplicar projeto.	P1
RF-005	Usuário deve poder excluir projeto.	P1
RF-006	Sistema deve armazenar cena como JSON versionado.	P0
RF-007	Sistema deve permitir abrir projetos recentes.	P1
RF-008	Sistema deve validar compatibilidade de cenas antigas com versões novas do runtime.	P1

8.2 Canvas e Editor Visual

ID	Requisito	Prioridade
RF-009	Editor deve exibir canvas WebGL em tempo real.	P0
RF-010	Usuário deve poder selecionar camada diretamente no canvas.	P0
RF-011	Usuário deve poder mover, escalar e rotacionar camadas.	P0
RF-012	Editor deve exibir guias, bounding boxes e handles de transformação.	P1
RF-013	Editor deve suportar zoom e pan do canvas.	P0
RF-014	Editor deve suportar diferentes proporções de tela.	P0
RF-015	Editor deve permitir pré-visualização em desktop, tablet e mobile.	P1
RF-016	Editor deve permitir alternar qualidade de preview.	P1

8.3 Sistema de Camadas

ID	Requisito	Prioridade
RF-017	Usuário deve poder adicionar camadas.	P0
RF-018	Usuário deve poder remover camadas.	P0
RF-019	Usuário deve poder reordenar camadas.	P0
RF-020	Usuário deve poder duplicar camadas.	P0
RF-021	Usuário deve poder ocultar/exibir camadas.	P0

ID	Requisito	Prioridade
RF-022	Usuário deve poder bloquear camadas.	P1
RF-023	Usuário deve poder agrupar camadas.	P1
RF-024	Usuário deve poder renomear camadas.	P0
RF-025	Camadas devem possuir blend modes.	P0
RF-026	Camadas devem possuir opacidade individual.	P0
RF-027	Camadas devem possuir resolução de renderização individual.	P0
RF-028	Camadas devem poder ser usadas como máscara.	P1

8.4 Tipos de Camada

ID	Tipo	Descrição	Prioridade
RF-029	Solid	Camada de cor sólida.	P0
RF-030	Shape	Formas básicas: retângulo, círculo, linha.	P0
RF-031	Image	Imagem importada pelo usuário.	P0
RF-032	Video	Vídeo importado pelo usuário.	P1
RF-033	Text	Texto editável.	P0
RF-034	Shader	Camada generativa baseada em shader.	P0
RF-035	3D Model	Modelo 3D simples, como GLB/GLTF.	P2

8.5 Biblioteca de Efeitos

ID	Requisito	Prioridade
RF-036	Usuário deve poder adicionar efeito a uma camada.	P0
RF-037	Usuário deve poder remover efeito de uma camada.	P0
RF-038	Usuário deve poder reordenar efeitos dentro da camada.	P1
RF-039	Cada efeito deve expor parâmetros editáveis.	P0
RF-040	Parâmetros devem aceitar valores numéricos, cor, booleanos e enums.	P0
RF-041	Parâmetros devem suportar reset para valor padrão.	P1
RF-042	Efeitos devem possuir presets.	P1

ID	Requisito	Prioridade
RF-043	Sistema deve permitir busca por nome de efeito.	P1
RF-044	Sistema deve categorizar efeitos.	P1

8.6 Categorias de Efeitos

Distortions

- FBM Distortion;
- Swirl;
- Polar;
- Wave;
- Ripple;
- Displacement;
- Lens Warp;
- Pixel Sort;
- Glitch Offset.

Generative

- Noise Fill;
- Wisps;
- Aurora;
- Gradient Field;
- Particles;
- Flow Field;
- Plasma;
- Organic Clouds.

Stylized / Post-processing

- Bloom;
- CRT;
- Dither;
- Grain;
- Chromatic Aberration;
- Blur;
- Pixelate;
- Posterize;
- Halftone;
- Color Shift;
- Vignette.

Compositing

- Mask;
 - Alpha Matte;
 - Luma Matte;
 - Blend;
 - Threshold;
 - Invert;
 - Color Replace.
-

8.7 Interatividade

ID	Requisito	Prioridade
RF-045	Propriedades devem poder reagir à posição do mouse.	P0
RF-046	Propriedades devem poder reagir à velocidade do mouse.	P1
RF-047	Propriedades devem poder reagir ao scroll.	P0
RF-048	Propriedades devem poder reagir ao tempo.	P0
RF-049	SDK deve permitir alterar variáveis em runtime.	P0
RF-050	Usuário deve poder mapear variável para parâmetro visualmente.	P1
RF-051	Usuário deve poder definir intensidade da influência interativa.	P0
RF-052	Usuário deve poder inverter, suavizar ou limitar valores de entrada.	P1

8.8 Animação

ID	Requisito	Prioridade
RF-053	Usuário deve poder animar propriedades numéricas.	P0
RF-054	Usuário deve poder definir duração da animação.	P0
RF-055	Usuário deve poder definir delay.	P0
RF-056	Usuário deve poder escolher easing.	P0
RF-057	Usuário deve poder criar loops.	P1
RF-058	Usuário deve poder sincronizar animação com scroll.	P1
RF-059	Sistema deve suportar animações de entrada e saída.	P0
RF-060	Timeline básica deve exibir propriedades animadas.	P1

8.9 Randomização Criativa

ID	Requisito	Prioridade
RF-061	Usuário deve poder randomizar propriedades da camada selecionada por atalho.	P1
RF-062	Atalho padrão sugerido: tecla S.	P1
RF-063	Usuário deve poder travar parâmetros que não devem ser randomizados.	P1
RF-064	Usuário deve poder definir intensidade da randomização.	P1
RF-065	Sistema deve permitir desfazer randomização.	P0
RF-066	Sistema deve registrar seed de randomização para resultados reproduzíveis.	P1

8.10 Máscaras e Composição

ID	Requisito	Prioridade
RF-067	Usuário deve poder usar uma camada como máscara de outra.	P1
RF-068	Máscaras devem suportar alpha matte.	P1
RF-069	Máscaras devem suportar luma matte.	P2
RF-070	Usuário deve poder inverter máscara.	P1
RF-071	Usuário deve poder controlar feather/blur da máscara.	P1
RF-072	Texto deve poder funcionar como máscara.	P1
RF-073	Formas devem poder funcionar como máscara.	P1

8.11 Flattening

ID	Requisito	Prioridade
RF-074	Sistema deve identificar camadas elegíveis para flattening.	P0
RF-075	Sistema deve combinar camadas compatíveis em um shader otimizado.	P1
RF-076	Sistema deve preservar resultado visual após flattening.	P0
RF-077	Sistema deve exibir diferença estimada de performance antes/depois.	P1
RF-078	Sistema deve permitir desativar flattening manualmente.	P1
RF-079	Sistema deve invalidar flattening quando parâmetros críticos mudarem.	P0

ID	Requisito	Prioridade
RF-080	Sistema deve salvar versão flattened na cena publicada.	P0
RF-081	Sistema deve fazer fallback para pipeline sequencial quando flattening falhar.	P0

8.12 Downsampling

ID	Requisito	Prioridade
RF-082	Camada deve permitir renderização em 100%, 75%, 50% e 25%.	P0
RF-083	Sistema deve sugerir downsampling para efeitos pesados.	P1
RF-084	Preview deve mostrar impacto visual do downsampling.	P0
RF-085	Exportação deve preservar configuração de downsampling.	P0
RF-086	SDK deve aplicar downsampling em produção.	P0

8.13 Estimador de Performance

ID	Requisito	Prioridade
RF-087	Editor deve exibir FPS atual.	P0
RF-088	Editor deve exibir tempo de renderização por frame.	P0
RF-089	Editor deve exibir score de custo entre 0 e 100.	P1
RF-090	Editor deve destacar camadas mais custosas.	P1
RF-091	Atalho sugerido para painel de performance: tecla F.	P1
RF-092	Sistema deve alertar quando cena exceder limite recomendado.	P1
RF-093	Sistema deve sugerir otimizações automáticas.	P2

8.14 Exportação

ID	Requisito	Prioridade
RF-094	Usuário deve poder exportar imagem estática.	P0
RF-095	Usuário deve poder exportar vídeo.	P1
RF-096	Usuário deve poder copiar embed code.	P0

ID	Requisito	Prioridade
RF-097	Usuário deve poder baixar JSON da cena.	P0
RF-098	Usuário deve poder publicar cena em CDN.	P1
RF-099	Exportação deve preservar flattening.	P0
RF-100	Exportação deve preservar downsampling.	P0
RF-101	Exportação deve preservar variáveis de runtime.	P0
RF-102	Sistema deve gerar snippet para Framer.	P1
RF-103	Sistema deve gerar snippet para React/Next.js.	P1

9. Requisitos Não Funcionais

9.1 Performance

- Editor deve manter 60 FPS em cenas simples.
- Editor deve manter no mínimo 30 FPS em cenas médias.
- Runtime publicado deve carregar rapidamente em sites externos.
- SDK inicial deve mirar tamanho inferior a 60kb gzipped, excluindo assets do usuário.
- Cena publicada deve iniciar renderização em menos de 1 segundo após carregamento dos assets principais em conexão razoável.
- Sistema deve degradar qualidade quando necessário em dispositivos mais fracos.

9.2 Compatibilidade

Browsers prioritários:

- Chrome;
- Safari;
- Edge;
- Firefox.

Dispositivos prioritários:

- desktop moderno;
- notebooks com GPU integrada;
- tablets;
- smartphones intermediários e premium.

Requisito técnico:

- WebGL2 como padrão.
- Fallback limitado para WebGL1 somente se viável.
- Detecção de suporte gráfico antes de carregar a cena.

9.3 Escalabilidade

- Projetos devem ser armazenados de forma versionada.
- Assets devem ser hospedados em storage/CDN.
- Runtime deve conseguir carregar cenas independentes por URL.
- Sistema deve suportar crescimento da biblioteca de efeitos.

9.4 Segurança

- Uploads devem validar tipo e tamanho de arquivo.
- Arquivos devem ser sanitizados.
- Embed não deve permitir execução arbitrária de scripts do usuário.
- JSON da cena deve ser validado antes de renderizar.
- SDK deve evitar exposição de tokens ou dados privados.
- Projetos privados não devem ser acessíveis sem permissão.

9.5 Confiabilidade

- Editor deve ter autosave.
- Alterações devem ser recuperáveis após crash.
- Flattening deve ter fallback seguro.
- Exportação não deve corromper o projeto original.
- Sistema deve avisar quando um asset não carregar.

9.6 Acessibilidade

- UI do editor deve ser navegável por teclado onde fizer sentido.
- Campos de parâmetros devem ter labels claros.
- Atalhos devem ser configuráveis no futuro.
- Contraste da interface deve seguir boas práticas de acessibilidade.

10. Experiência do Usuário

10.1 Estrutura da Interface

A interface deve ser organizada em quatro áreas principais:

1. Top Bar

2. nome do projeto;
3. botão salvar;
4. undo/redo;
5. botão preview;
6. botão exportar/publicar;
7. indicador de performance.

8. Canvas Central

9. visualização WebGL em tempo real;

10. zoom e pan;

11. seleção visual de camadas;

12. handles de transformação;

13. guias e grid opcionais.

14. Painel Esquerdo — Camadas

15. lista ordenada de camadas;

16. adicionar camada;

17. duplicar;

18. excluir;

19. agrupar;

20. visibilidade;

21. lock;

22. blend mode;

23. uso como máscara.

24. Painel Direito — Propriedades

25. transform;

26. aparência;

27. efeitos;

28. animação;

29. interatividade;

30. performance;

31. export settings.

10.2 Fluxo Principal

1. Usuário cria novo projeto.

2. Escolhe tamanho da cena.

3. Adiciona camada base.

4. Aplica efeitos.

5. Ajusta parâmetros.

6. Adiciona interatividade.

7. Testa preview.

8. Verifica performance.

9. Aplica otimizações.

10. Exporta embed ou imagem/vídeo.

10.3 Princípios de UX

- A experiência deve ser visual antes de técnica.
 - O usuário deve conseguir experimentar rapidamente.
 - Toda mudança relevante deve aparecer em tempo real.
 - Parâmetros técnicos devem ter nomes amigáveis.
 - O produto deve incentivar exploração criativa.
 - A performance deve ser compreensível, não assustadora.
 - Erros técnicos devem ser traduzidos em mensagens acionáveis.
-

11. Arquitetura Técnica

11.1 Visão Geral

A arquitetura será composta por:

1. Editor Web

2. aplicação React/Next.js;
3. UI de edição;
4. gerenciamento de projeto;
5. canvas WebGL;
6. painéis e controles.

7. Render Engine

8. motor WebGL2;
9. sistema de camadas;
10. gerenciamento de shaders;
11. render targets;
12. composição;
13. máscaras;
14. interatividade.

15. Scene Compiler

16. serialização;
17. validação;
18. otimização;
19. flattening;

20. geração de pacote para runtime.

21. Runtime SDK

22. renderização standalone;

23. carregamento de cena;

24. API pública;

25. controle de variáveis;

26. lifecycle hooks.

27. Backend

28. autenticação;

29. projetos;

30. assets;

31. publicação;

32. CDN;

33. versionamento.

12. Modelo de Renderização

12.1 Pipeline Padrão

Cada camada é processada individualmente. A saída de uma camada é enviada para um render target. A camada seguinte recebe essa saída como entrada e aplica sua própria composição.

Fluxo:

```
Layer 1 → Render Target A
Layer 2 + A → Render Target B
Layer 3 + B → Render Target C
Final Output → Canvas
```

12.2 Estrutura de Camada

Cada camada deve conter:

```
{
  "id": "layer_123",
  "type": "shader",
```

```

    "name": "Aurora Background",
    "visible": true,
    "locked": false,
    "opacity": 1,
    "blendMode": "screen",
    "transform": {
      "x": 0,
      "y": 0,
      "scaleX": 1,
      "scaleY": 1,
      "rotation": 0
    },
    "effects": [],
    "animation": {},
    "interactions": {},
    "performance": {
      "resolutionScale": 1,
      "flattenEligible": true
    }
  }
}

```

12.3 Blend Modes Iniciais

O MVP deve suportar:

- Normal;
- Multiply;
- Screen;
- Add;
- Overlay;
- Difference;
- Lighten;
- Darken.

13. Sistema de Shaders

13.1 Conceito

Cada efeito será implementado como um módulo de shader com:

- fragment shader;
- vertex shader, quando necessário;
- uniforms;
- parâmetros expostos para UI;
- valores padrão;

- range mínimo/máximo;
- categoria;
- custo estimado.

13.2 Estrutura de Efeito

```
{
  "id": "fbm_distortion",
  "name": "FBM Distortion",
  "category": "Distortions",
  "inputs": ["sourceTexture", "uv", "time"],
  "uniforms": {
    "strength": {
      "type": "float",
      "default": 0.25,
      "min": 0,
      "max": 1
    },
    "scale": {
      "type": "float",
      "default": 4,
      "min": 0.1,
      "max": 20
    },
    "speed": {
      "type": "float",
      "default": 0.5,
      "min": 0,
      "max": 5
    }
  },
  "cost": 14,
  "flattenCompatible": true
}
```

13.3 Regras de Compatibilidade

Um efeito pode ser flatten-compatible quando:

- não depende de feedback de frame anterior;
- não usa múltiplos passes obrigatórios;
- não depende de leitura dinâmica de render target externo;
- não usa vídeo com tempo independente não determinístico;
- não depende de assets ainda não resolvidos;
- não possui side effects no pipeline.

14. Flattening

14.1 Objetivo

O flattening é o sistema que transforma uma pilha de camadas e efeitos compatíveis em um shader otimizado único ou em menos passes de renderização.

Isso reduz:

- draw calls;
- alocação de render targets;
- leituras e escritas de textura;
- custo de composição;
- consumo de memória GPU;
- tempo de renderização por frame.

14.2 Funcionamento

O Scene Compiler deve:

1. Analisar grafo de camadas.
2. Identificar blocos contínuos compatíveis.
3. Verificar blend modes, máscaras, dependências e efeitos.
4. Gerar um shader combinado.
5. Remapear uniforms.
6. Gerar fallback.
7. Comparar output visual em preview, quando possível.
8. Salvar versão otimizada da cena publicada.

14.3 Exemplo

Antes:

```
Noise Layer → RT1  
Swirl Effect → RT2  
Color Shift → RT3  
Vignette → RT4  
Final Composite
```

Depois:

```
Combined Shader → Final Output
```

14.4 Tipos de Flattening

Flattening Local

Combina efeitos dentro de uma única camada.

Flattening de Grupo

Combina múltiplas camadas dentro de um grupo compatível.

Flattening de Cena

Combina a maior parte da cena em um único shader final, quando possível.

14.5 Culling

O compilador deve descartar computação de pixels que serão invisíveis por:

- opacidade zero;
- máscara totalmente preta;
- camada fora do viewport;
- área totalmente coberta por camada opaca superior;
- escala efetiva irrelevante.

14.6 Hoisting de Distorções

Quando possível, distorções globais devem ser elevadas no pipeline para evitar repetição desnecessária em múltiplas camadas.

Exemplo:

```
Layer A + Distortion  
Layer B + Distortion  
Layer C + Distortion
```

Pode virar:

```
Composite A/B/C → Single Distortion
```

Desde que o resultado visual seja preservado ou a diferença seja aceitável.

14.7 Critérios de Aceitação do Flattening

- Resultado visual deve ser equivalente em pelo menos 95% dos casos compatíveis.
- O usuário deve poder desativar flattening caso perceba diferença visual.

- Sistema deve fazer fallback automaticamente em caso de erro.
 - Cenas publicadas devem carregar com a versão otimizada.
 - Editor deve mostrar quando uma camada está otimizada.
-

15. Downsampling

15.1 Objetivo

Permitir que camadas custosas sejam renderizadas em resolução menor, reduzindo o número de pixels processados.

15.2 Configurações

Cada camada deve aceitar:

- 100%;
- 75%;
- 50%;
- 25%;
- Auto.

15.3 Modo Auto

O sistema pode sugerir ou aplicar downsampling automático baseado em:

- tipo de efeito;
- área ocupada na tela;
- blur posterior;
- velocidade de animação;
- ruído orgânico;
- custo estimado;
- capacidade do dispositivo.

15.4 Critérios de Aceitação

- Downsampling deve preservar proporção da camada.
 - Preview deve representar fielmente o resultado publicado.
 - Usuário deve conseguir comparar antes/depois.
 - SDK deve aplicar a mesma configuração em produção.
-

16. Estimador de Performance

16.1 Métricas

O painel de performance deve exibir:

- FPS;
- frame time;
- draw calls;
- número de render targets;
- memória estimada de textura;
- custo total da cena;
- custo por camada;
- custo por efeito;
- status de flattening;
- resolução efetiva por camada.

16.2 Score de Custo

Score de 0 a 100:

- 0–30: leve;
- 31–60: moderado;
- 61–80: pesado;
- 81–100: crítico.

16.3 Fórmula Inicial Sugerida

```
cost = shaderCost
      + textureReadCost
      + renderTargetCost
      + resolutionCost
      + animationCost
      + interactionCost
      - flatteningSavings
```

A fórmula deve evoluir com dados reais coletados em testes.

16.4 Recomendações Automáticas

O sistema deve sugerir:

- reduzir resolução da camada;
- aplicar flattening;
- remover efeito caro;
- diminuir blur/bloom;

- reduzir quantidade de camadas;
- trocar vídeo por imagem;
- limitar animações em mobile.

17. Runtime SDK

17.1 Objetivo

Permitir que cenas criadas no editor sejam incorporadas em sites externos com baixo custo de carregamento e boa performance.

17.2 Requisitos

- SDK JavaScript leve.
- Carregamento por URL ou JSON inline.
- API para inicializar cena.
- API para destruir cena.
- API para pausar/resumir.
- API para atualizar variáveis.
- API para reagir a eventos.
- Suporte a resize responsivo.
- Suporte a DPR controlado.
- Suporte a fallback image.

17.3 Exemplo de Uso

```
<div id="scene"></div>

<script src="https://cdn.product.com/sdk.min.js"></script>
<script>
  const scene = ShaderCanvas.create({
    container: "#scene",
    sceneUrl: "https://cdn.product.com/scenes/abc123.json",
    autoplay: true
  });

  scene.setVariable("intensity", 0.8);
  scene.setVariable("scrollProgress", 0.4);
</script>
```

17.4 API Inicial

```
type ShaderCanvasInstance = {
  play(): void;
```

```
pause(): void;
destroy(): void;
resize(): void;
setVariable(name: string, value: number | string | boolean): void;
getVariable(name: string): unknown;
on(event: string, callback: Function): void;
off(event: string, callback: Function): void;
};
```

18. Exportação e Publicação

18.1 Exportar Imagem

Usuário pode exportar:

- PNG;
- JPEG;
- resolução customizada;
- fundo transparente, quando possível.

18.2 Exportar Vídeo

Usuário pode exportar:

- WebM;
- MP4, se suportado;
- duração;
- FPS;
- resolução;
- loop perfeito, quando possível.

18.3 Embed

Usuário pode copiar:

- script embed simples;
- código React;
- código Next.js;
- snippet para Framer;
- iframe, se aplicável.

18.4 Publicação CDN

Cena publicada deve conter:

- JSON otimizado;

- assets comprimidos;
 - shaders compiláveis;
 - metadados de versão;
 - fallback visual;
 - configurações de performance.
-

19. Modelo de Dados

19.1 Projeto

```
{
  "id": "project_123",
  "name": "Landing Page Hero",
  "ownerId": "user_123",
  "createdAt": "2026-01-01T00:00:00Z",
  "updatedAt": "2026-01-01T00:00:00Z",
  "scene": {},
  "publishedVersion": "version_123"
}
```

19.2 Cena

```
{
  "version": "1.0.0",
  "canvas": {
    "width": 1440,
    "height": 900,
    "background": "#000000",
    "dpr": "auto"
  },
  "layers": [],
  "variables": [],
  "performance": {
    "flattening": true,
    "targetFPS": 60
  }
}
```

19.3 Asset

```
{
  "id": "asset_123",
  "type": "image",
}
```

```
"url": "https://cdn.product.com/assets/image.png",
"width": 1920,
"height": 1080,
"size": 1240000,
"mimeType": "image/png"
}
```

20. Backend e Infraestrutura

20.1 Stack Sugerida

Front-end

- Next.js;
- React;
- TypeScript;
- Zustand ou Jotai para estado local;
- TanStack Query para dados remotos;
- Tailwind ou CSS modular;
- WebGL2 custom engine.

Backend

- Node.js;
- PostgreSQL;
- Prisma;
- S3-compatible storage;
- CDN;
- Auth.js, Clerk ou Supabase Auth.

Render Engine

- TypeScript;
- WebGL2;
- GLSL;
- sistema próprio de shader modules;
- worker para compilação/otimização quando possível.

20.2 Serviços

- Auth;
- Project API;
- Asset API;
- Publish API;
- Scene Compiler;
- CDN Delivery;

- Analytics/Telemetry;
 - Billing.
-

21. Telemetria e Analytics

21.1 Métricas de Produto

- projetos criados;
- cenas publicadas;
- embeds ativos;
- exportações realizadas;
- efeitos mais usados;
- templates mais usados;
- taxa de conclusão de onboarding;
- usuários ativos semanais;
- conversão free → paid.

21.2 Métricas Técnicas

- FPS médio no editor;
- frame time médio no runtime;
- taxa de erro de shader compile;
- taxa de fallback WebGL;
- tempo de carregamento do SDK;
- peso médio de cena publicada;
- uso médio de memória GPU;
- falhas de exportação.

21.3 Eventos Relevantes

```
project_created  
layer_added  
effect_added  
effect_randomized  
scene_previewed  
performance_panel_opened  
flattening_enabled  
downsampling_changed  
scene_exported  
scene_published  
embed_copied  
sdk_loaded  
runtime_error
```

22. Monetização

22.1 Plano Free

- limite de projetos;
- watermark opcional em exportações;
- limite de cenas publicadas;
- biblioteca básica de efeitos;
- exportação limitada.

22.2 Plano Pro

- projetos ilimitados;
- publicação sem watermark;
- biblioteca completa de efeitos;
- exportação de vídeo;
- templates premium;
- SDK avançado;
- controle de variáveis;
- CDN incluída.

22.3 Plano Studio

- múltiplos membros;
 - biblioteca compartilhada;
 - gestão de clientes;
 - templates privados;
 - permissões;
 - histórico de versões;
 - suporte prioritário.
-

23. Critérios de Sucesso

23.1 Sucesso do MVP

O MVP será considerado bem-sucedido se:

- usuários conseguirem criar uma cena visualmente interessante em menos de 10 minutos;
- usuários conseguirem publicar uma cena em site externo sem suporte técnico;
- cenas simples rodarem a 60 FPS em desktop;
- cenas médias rodarem a pelo menos 30 FPS em notebooks comuns;
- SDK for incorporado com menos de 5 linhas de código;
- usuários entenderem o painel de performance;
- pelo menos 30% dos usuários testarem randomização ou presets;
- pelo menos 20% dos projetos criados forem exportados ou publicados.

23.2 Métricas Norteadoras

North Star Metric

Número de cenas WebGL publicadas com sucesso por semana.

Métricas Secundárias

- tempo até primeira cena publicada;
 - número médio de camadas por cena;
 - número médio de efeitos por cena;
 - taxa de exportação;
 - taxa de erro na publicação;
 - performance média das cenas publicadas;
 - retenção semanal.
-

24. Regras de Qualidade

24.1 Qualidade Visual

- Efeitos devem ter boa aparência com valores padrão.
- Presets devem gerar resultados úteis sem ajuste manual.
- Randomização deve evitar resultados quebrados.
- Blend modes devem ser previsíveis.
- Texto deve permanecer legível quando usado como camada comum.

24.2 Qualidade Técnica

- Shaders devem compilar de forma consistente.
- Falhas de shader não devem derrubar o editor.
- Assets grandes devem ser otimizados.
- Runtime deve limpar recursos GPU ao destruir cena.
- Cena publicada deve ser determinística.

24.3 Qualidade de UX

- Usuário deve sempre entender qual camada está selecionada.
 - Alterações devem ter feedback visual imediato.
 - Erros devem explicar o que fazer.
 - Atalhos devem ser documentados.
 - Performance deve ser apresentada de forma prática.
-

25. Riscos

25.1 Riscos Técnicos

Risco	Impacto	Mitigação
Flattening ser complexo demais para MVP	Alto	Implementar flattening local primeiro
Diferença visual após otimização	Alto	Permitir fallback e toggle manual
Performance ruim em mobile	Alto	Downsampling, DPR controlado e presets mobile
Shaders falharem entre browsers	Médio	Testes cross-browser e fallback
Exportação de vídeo instável	Médio	Priorizar imagem e embed no MVP
SDK ficar pesado	Alto	Separar editor e runtime rigorosamente

25.2 Riscos de Produto

Risco	Impacto	Mitigação
Ferramenta parecer técnica demais	Alto	UX orientada a presets e controles simples
Usuário não entender performance	Médio	Score visual e recomendações
Biblioteca inicial parecer limitada	Médio	Criar efeitos de alta qualidade no MVP
Concorrência com ferramentas estabelecidas	Alto	Diferenciar por performance e embed
Usuários criarem cenas pesadas demais	Alto	Alertas e otimização automática

25.3 Riscos Legais e de Propriedade Intelectual

- Não copiar marca, interface, textos, código, assets ou shaders proprietários de concorrentes.
- Criar biblioteca própria de efeitos.
- Usar nomenclatura genérica quando apropriado.
- Documentar autoria dos shaders.
- Garantir licenças corretas para assets, fontes e bibliotecas.

26. Roadmap

Fase 0 — Pesquisa e Prototipação

Duração estimada: 2 a 4 semanas.

Entregas:

- protótipo WebGL de camadas;
- render targets sequenciais;
- 3 efeitos básicos;
- importação de imagem;
- teste inicial de performance;
- prova de conceito de embed.

Fase 1 — MVP Técnico

Duração estimada: 6 a 10 semanas.

Entregas:

- editor básico;
- sistema de camadas;
- painel de propriedades;
- 12 a 20 efeitos;
- animação básica;
- interatividade com mouse/time/scroll;
- exportação JSON;
- SDK inicial;
- downsampling manual.

Fase 2 — MVP Produto

Duração estimada: 6 a 8 semanas.

Entregas:

- autenticação;
- dashboard;
- salvar projetos;
- upload de assets;
- publicação;
- embed code;
- presets;
- performance panel;
- templates iniciais;
- onboarding.

Fase 3 — Otimização Avançada

Duração estimada: 8 a 12 semanas.

Entregas:

- flattening local;
- flattening de grupos;
- score de custo;
- recomendações automáticas;
- suporte mobile melhorado;
- CDN otimizada;
- fallback image;
- exportação de vídeo.

Fase 4 — Comercialização

Duração estimada: contínua.

Entregas:

- billing;
- planos Free/Pro/Studio;
- landing page;
- documentação;
- exemplos;
- galeria pública;
- templates premium;
- suporte a times.

27. Critérios de Aceitação do MVP

27.1 Editor

- Usuário consegue criar projeto novo.
- Usuário consegue adicionar pelo menos 5 tipos de camada.
- Usuário consegue aplicar efeitos.
- Usuário consegue editar parâmetros.
- Usuário consegue reordenar camadas.
- Usuário consegue visualizar resultado em tempo real.
- Undo/redo funciona para ações principais.

27.2 Renderização

- Cena simples roda a 60 FPS em desktop moderno.
- Cena média roda a pelo menos 30 FPS.
- Sistema suporta pelo menos 10 camadas simples.
- Sistema suporta pelo menos 5 efeitos simultâneos.
- Render target pipeline funciona sem vazamento de memória.

27.3 Exportação

- Usuário consegue exportar imagem.
- Usuário consegue copiar embed code.
- Cena embedada roda fora do editor.
- SDK inicializa, pausa, resume e destrói cena.
- Runtime aplica variáveis externas.

27.4 Performance

- Painel exibe FPS.
 - Painel exibe frame time.
 - Downsampling altera custo visualmente.
 - Cena publicada preserva configuração de performance.
 - Sistema alerta em caso de cena pesada.
-

28. Onboarding

28.1 Primeiro Uso

O onboarding deve guiar o usuário por:

1. escolher um template;
2. selecionar uma camada;
3. alterar um parâmetro;
4. aplicar um efeito;
5. randomizar valores;
6. testar interatividade;
7. abrir painel de performance;
8. exportar ou publicar.

28.2 Templates Iniciais

- Aurora Background;
 - Interactive Noise Hero;
 - Distorted Image;
 - Liquid Gradient;
 - Text Mask Reveal;
 - CRT Poster;
 - Organic Wisps;
 - Scroll Reactive Background.
-

29. Documentação

29.1 Documentação para Usuário

- Como criar uma cena.
- Como usar camadas.
- Como aplicar efeitos.
- Como usar máscaras.
- Como animar propriedades.
- Como usar interatividade.
- Como otimizar performance.
- Como publicar no Framer.
- Como publicar no Webflow.
- Como usar embed.

29.2 Documentação para Desenvolvedor

- Instalação do SDK.
- API de runtime.
- Controle de variáveis.
- Eventos.
- Lifecycle.
- Performance.
- Fallbacks.
- Exemplos em React/Next.js.
- Troubleshooting WebGL.

30. Decisões Técnicas Recomendadas

30.1 Separar Editor e Runtime

O editor pode ser pesado, com UI, ferramentas e debug. O runtime precisa ser mínimo e focado apenas em carregar e renderizar cenas.

30.2 Scene JSON como Fonte da Verdade

Toda cena deve ser representada em JSON versionado. Isso facilita:

- salvar projetos;
- publicar;
- migrar versões;
- gerar runtime;
- debugar;
- criar templates;
- compartilhar cenas.

30.3 Sistema Modular de Efeitos

Cada efeito deve ser autocontido, com metadados suficientes para UI, renderização, performance e flattening.

30.4 Otimização Progressiva

Não tentar resolver flattening completo no início. Começar com:

1. downsampling manual;
2. flattening dentro da camada;
3. flattening de grupos simples;
4. flattening avançado de cena.

30.5 Presets como Parte Central da UX

Usuários criativos precisam de resultados rápidos. Presets devem ser tratados como feature principal, não detalhe secundário.

31. Perguntas em Aberto

1. O produto terá foco maior em designers ou desenvolvedores?
 2. O MVP precisa ter colaboração entre usuários?
 3. A publicação será sempre via CDN própria ou também via export self-hosted?
 4. O produto precisa funcionar offline?
 5. Exportação de vídeo é essencial no MVP ou pode ser V1?
 6. O sistema terá billing desde o início?
 7. Haverá biblioteca pública de templates?
 8. O editor permitirá importar arquivos Figma/SVG no futuro?
 9. O produto terá suporte a WebGPU em roadmap próximo?
 10. Qual será o limite de tamanho de assets por plano?
-

32. Priorização Recomendada

P0 — Essencial para MVP

- Canvas WebGL.
- Sistema de camadas.
- Painel de propriedades.
- Biblioteca inicial de efeitos.
- Interatividade básica.
- Animação básica.
- Export JSON.
- Embed code.

- SDK runtime.
- Downsampling.
- FPS/frame time.
- Salvar projeto.

P1 — Importante para V1

- Máscaras.
- Presets.
- Templates.
- Flattening básico.
- Score de performance.
- Export vídeo.
- Framer/Webflow snippets.
- Autenticação completa.
- Publicação CDN.
- Histórico simples.

P2 — Futuro

- Colaboração.
- Marketplace.
- Node editor.
- WebGPU.
- Analytics avançado.
- Plugins.
- Modelos 3D avançados.
- Automação via API.

33. Conclusão

Este produto tem potencial para ocupar um espaço relevante entre ferramentas de design visual, motion design e desenvolvimento criativo para web.

O diferencial não está apenas em permitir criar efeitos WebGL sem código, mas em resolver o problema completo: criação visual, controle criativo, interatividade, publicação e performance.

O ponto mais importante do projeto é evitar que a ferramenta vire apenas um brinquedo visual pesado. Para ser realmente útil em produção, ela precisa tratar performance como parte central da experiência. Por isso, recursos como downsampling, estimador de custo, flattening e SDK leve devem ser considerados pilares do produto.

A recomendação é iniciar com um MVP focado em:

1. editor visual simples;
2. camadas e efeitos de alta qualidade;

3. embed funcional;
4. performance aceitável;
5. fundação técnica preparada para flattening avançado.

Com essa base, o produto pode evoluir para uma plataforma criativa robusta para designers, devs e estúdios que querem produzir experiências visuais interativas modernas sem depender de programação shader manual.